

# Matriz e lista de adjacência(GRAFOS)

Quando trabalhamos com **grafos**, precisamos de uma forma de armazenar os vértices e as conexões (arestas). As duas representações mais utilizadas são:

**Matriz de Adjacência**

**Lista de Adjacência**

# Matriz e lista de adjacência(GRAFOS)

## O que é um Grafo?

Imagine as seguintes cidades conectadas por estradas:

```
graph TD; A --- B; C --- D; A --- C; B --- D;
```

A diagram showing four nodes labeled A, B, C, and D arranged in a square. A and B are at the top, C and D are at the bottom. A and C are on the left, B and D are on the right. There are dashed lines connecting A to B and C to D. There are vertical lines connecting A to C and B to D.

# Matriz e lista de adjacência(GRAFOS)

Podemos representar isso como:

A -> B

A -> C

B -> D

C -> D

Onde:

A, B, C e D são os **vértices**

As ligações são as **arestas**

# Matriz e lista de adjacência(GRAFOS)

## 1. Matriz de Adjacência

Uma matriz de adjacência é uma tabela onde:

Linhas representam a origem.

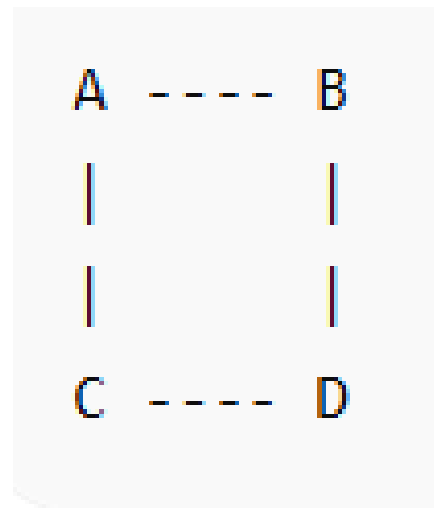
Colunas representam o destino.

Valor 1 significa que existe conexão.

Valor 0 significa que não existe conexão.

### Exemplo

Considere o grafo:



# Matriz e lista de adjacência(GRAFOS)

Numerando os vértices:

A = 0

B = 1

C = 2

D = 3

A matriz fica:

|   | A | B | C | D |
|---|---|---|---|---|
| A | 0 | 1 | 1 | 0 |
| B | 1 | 0 | 0 | 1 |
| C | 1 | 0 | 0 | 1 |
| D | 0 | 1 | 1 | 0 |

OU:

A B C D

A -> [0 1 1 0]

B -> [1 0 0 1]

C -> [1 0 0 1]

D -> [0 1 1 0]

# Matriz e lista de adjacência(GRAFOS)

```
#include <stdio.h>
#define V 4
int main() {
    int grafo[V][V] = {
        {0,1,1,0},
        {1,0,0,1},
        {1,0,0,1},
        {0,1,1,0}
    };
    printf("Matriz de Adjacencia:\n\n");
    for(int i=0;i<V;i++) {
        for(int j=0;j<V;j++) {
            printf("%d ", grafo[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

# Matriz e lista de adjacência(GRAFOS)

## Saída

```
0  1  1  0
1  0  0  1
1  0  0  1
0  1  1  0
```

## Vantagens da Matriz

- ✓ Fácil implementação.
- ✓ Verificar se existe uma aresta é muito rápido:

```
if (grafo[0][1] == 1)
```

# Matriz e lista de adjacência(GRAFOS)

## **Desvantagens da Matriz**

Se tivermos:

1000 vértices

Precisaremos:

$1000 \times 1000 = 1.000.000$  posições

Mesmo que existam poucas conexões.

Isso desperdiça memória.



# Matriz e lista de adjacência(GRAFOS)

## 2. Lista de Adjacência

Em vez de guardar todas as possíveis conexões, armazenamos apenas as que realmente existem.

### Mesmo Grafo

```
A  ----  B
|         |
|         |
C  ----  D
```

Representação:

A -> B -> C

B -> A -> D

C -> A -> D

D -> B -> C

# Matriz e lista de adjacência(GRAFOS)

## Visualização

0 -> 1 -> 2

1 -> 0 -> 3

2 -> 0 -> 3

3 -> 1 -> 2

# Matriz e lista de adjacência(GRAFOS)

```
#include <stdio.h>
#include <stdlib.h>
#define V 4

typedef struct No {
    int vertice;
    struct No* prox;
} No;

No* lista[V];

void adicionarAresta(int origem, int destino) {

    No* novo = (No*) malloc(sizeof(No));
    novo->vertice = destino;
    novo->prox = lista[origem];
    lista[origem] = novo;
}
```

# Matriz e lista de adjacência(GRAFOS)

```
void imprimirGrafo() {  
  
    for(int i=0;i<V;i++) {  
  
        printf("%d -> ", i);  
  
        No* temp = lista[i];  
  
        while(temp != NULL) {  
            printf("%d ", temp->vertice);  
            temp = temp->prox;  
        }  
  
        printf("\n");  
    }  
}
```

# Matriz e lista de adjacência(GRAFOS)

```
int main() {  
    for(int i=0;i<V;i++)  
        lista[i] = NULL;  
    adicionarAresta(0,1);  
    adicionarAresta(0,2);  
    adicionarAresta(1,0);  
    adicionarAresta(1,3);  
    adicionarAresta(2,0);  
    adicionarAresta(2,3);  
    adicionarAresta(3,1);  
    adicionarAresta(3,2);  
    imprimirGrafo();  
    return 0;  
}
```

# Matriz e lista de adjacência(GRAFOS)

## Saída

```
0 -> 2 1
1 -> 3 0
2 -> 3 0
3 -> 2 1
```

A ordem pode mudar porque estamos inserindo no início da lista.

## Comparação

| Característica     | Matriz  | Lista                       |
|--------------------|---------|-----------------------------|
| Implementação      | Simples | Mais complexa               |
| Uso de memória     | Alto    | Baixo                       |
| Verificar aresta   | $O(1)$  | $O(\text{grau do vértice})$ |
| Percorrer vizinhos | $O(V)$  | $O(\text{grau})$            |
| Grafos densos      | Melhor  | Boa                         |
| Grafos esparsos    | Ruim    | Melhor                      |

# Matriz e lista de adjacência(GRAFOS)

## **Exemplo de Consumo de Memória**

Suponha:

1000 vértices

2000 arestas

### **Matriz**

1000 x 1000

= 1.000.000 posições

### **Lista**

Armazena apenas:

2000 arestas

A economia de memória é enorme.

# Matriz e lista de adjacência(GRAFOS)

**Quando usar cada uma?**

**Use Matriz de Adjacência quando:**

O grafo é pequeno.

Existem muitas conexões.

Você precisa consultar rapidamente se uma aresta existe.

Exemplo:

Redes pequenas

Jogos simples

Exercícios acadêmicos



# Matriz e lista de adjacência(GRAFOS)

**Use Lista de Adjacência quando:**

O grafo é grande.

Existem poucas conexões.

Memória é importante.

Exemplo:

GPS

Google Maps

Redes Sociais

Internet

Algoritmo de Dijkstra com Heap